ESSENTIAL DATABASE OPERATIONS IN CHROMA DB

Professional Study Notes, Code Examples, Revision Summary, Assessment Questions and Brainstorming Answers

Focus: Collection lifecycle management, document CRUD operations, embeddings, metadata, HNSW indexing, and distance metrics.

Prepared as a structured learning resource for practical vector database and RAG development.

Learning Objectives

After completing these notes, you should be able to:

- Create and connect to collections in Chroma DB.
- Modify collection names and metadata.
- Add, retrieve, update, and delete documents.
- Explain how embeddings are generated and refreshed.
- Use IDs and metadata filters correctly.
- Describe the role of HNSW in approximate nearest-neighbor search.
- Select an appropriate distance metric for a vector-search workload.

Core idea
Chroma DB operations provide the data-management foundation for semantic search, retrieval-augmented generation (RAG), recommendation, and other vector-based applications.

1. Collections in Chroma DB

A collection is the primary organizational container in Chroma DB. It groups related documents, their vector embeddings, metadata, and unique identifiers so they can be managed and searched together.

Typical collection contents

- Documents: the original text or content.
- Embeddings: numerical vector representations of the content.
- Metadata: descriptive key-value information attached to a collection or document.
- IDs: unique identifiers used to address individual records.

Example use cases

- Technical documentation knowledge base
- Banking policies and operating procedures
- Product catalog and recommendation data
- Customer-support articles and FAQs
- Enterprise RAG content repository

2. Creating a Collection

Step 1: Import Chroma DB and embedding utilities

```
import chromadb
from chromadb.utils import embedding_functions
```

Step 2: Define the embedding model

```
embedding_function = (
    embedding_functions.SentenceTransformerEmbeddingFunction(
        model_name="all-MiniLM-L6-v2"
    )
)
```

The embedding function converts each document into a numerical vector. Those vectors enable semantic comparison between documents and queries.

Step 3: Create a client

```
client = chromadb.Client()
```

Step 4: Create the collection

```
collection = client.create_collection(
    name="my_collection",
    metadata={"description": "Demo collection"},
    embedding_function=embedding_function
)
```

Important parameters

Parameter	Purpose
name	Unique collection name.
metadata	Describes the purpose, ownership, version, or content.
embedding_function	Defines how text is converted into embeddings.

Metadata design tip
Use consistent keys such as owner, domain, source, version, environment, and sensitivity. Consistent metadata makes later filtering and governance easier.

3. Connecting to an Existing Collection

```
collection = client.get_collection(
    name="my_collection",
    embedding_function=embedding_function
)

print(collection.metadata)
```

The get_collection() method retrieves a previously created collection. Printing its metadata is a simple way to verify that the expected collection was opened.

Operational caution
Use the same compatible embedding function when reconnecting to a collection. A mismatch between ingestion and query-time embeddings can produce poor retrieval results.

4. Modifying a Collection

```
collection.modify(
    name="renamed_collection",
    metadata={"key": "value"}
)
```

What can be modified

- Collection name
- Collection-level metadata and descriptive attributes

What normally requires a new or cloned collection

- Embedding model or embedding dimensionality
- HNSW distance metric or vector-space configuration

These changes affect how every stored vector is produced or compared. Existing data therefore needs to be re-embedded or rebuilt. For large collections, cloning or migration can be

computationally expensive.

5. Adding Documents

```
collection.add(
    documents=[
        "LangChain is a framework for LLM applications.",
        "LlamaIndex supports data ingestion and retrieval."
    ],
    metadatas=[
        {"source": "docs", "version": "1.0", "topic": "LangChain"},
        {"source": "docs", "version": "1.0", "topic": "LlamaIndex"}
    ],
    ids=["id1", "id2"]
)
```

What happens during insertion

- The document text is stored.
- The embedding function converts the document into a vector.
- The embedding is indexed for similarity search.
- Metadata is stored with the record.
- The unique ID becomes the primary reference for later retrieval, update, or deletion.

Validation rule
The number of documents, metadata dictionaries, and IDs should align. IDs must be unique within the collection.

6. Retrieving Data

Retrieve all records

```
results = collection.get()
print(results)
```

The result is returned as a Python dictionary. It can include IDs, documents, metadata, and other requested fields.

Retrieve embeddings explicitly

```
results = collection.get(include=["embeddings"])
```

Embeddings are stored but are not normally displayed by default. Request them only when needed because vectors can make the response large.

Retrieve selected records by ID

```
results = collection.get(ids=["id1"])
```

ID-based retrieval is useful for validation, audit checks, troubleshooting, and record-level maintenance.

7. Updating Documents

```
collection.update(
    ids=["id1"],
    documents=["Updated LangChain content for production RAG."],
    metadatas=[{
        "source": "documentation",
        "version": "2.0",
        "topic": "LangChain"
    }]
)
```

The update() method changes one or more records by ID. If document text changes, Chroma DB re-computes the embedding so the vector index reflects the new content.

Update workflow

- Locate the record by its ID.
- Submit the revised document and/or metadata.
- Re-embed the updated text.
- Refresh the vector index.
- Verify the record using get().

> Best practice
> Treat updates as controlled data changes. Keep version, modified date, and source metadata where auditability matters.

8. Deleting Documents

Delete by ID

```
collection.delete(ids=["id1"])
```

Delete using a metadata filter

```
collection.delete(where={"topic": "LlamaIndex"})
```

Use IDs and filters together

IDs and metadata filters can be combined for more selective deletion. The record is deleted only when it falls within the supplied constraints.

> Production safeguard
> Before bulk deletion, run an equivalent retrieval/filter operation and review the matched IDs. This reduces the risk of deleting unintended records.

9. HNSW and Approximate Nearest-Neighbor Search

HNSW stands for Hierarchical Navigable Small World. It is an approximate nearest-neighbor algorithm used to locate vectors that are close to a query vector without exhaustively comparing the query against every stored vector.

Why HNSW is important

- Fast retrieval over large vector collections
- Practical balance between search speed and recall
- Supports semantic similarity search

- Scales better than exhaustive comparison for many workloads

The space setting determines the distance function used to compare embeddings.

10. Distance Metrics

Metric	Meaning	Typical use
l2	Squared Euclidean distance	General vector comparison; default in the lesson.
cosine	Angular similarity/distance	Common for text embeddings, semantic search, and RAG.
ip	Inner product or dot product	Ranking and recommendation workloads, depending on model training.

Configure the distance metric at collection creation

```
collection = client.create_collection(
    name="my_collection",
    metadata={"hnsw:space": "cosine"},
    embedding_function=embedding_function
)
```

Design decision
Choose the metric before large-scale ingestion. Changing it later generally requires creating or cloning a collection and rebuilding the index.

11. End-to-End Operational Flow

1	Define embedding function
2	Create or open a collection
3	Add documents, metadata, and IDs
4	Generate and index embeddings
5	Retrieve or query records
6	Update documents and re-embed
7	Delete obsolete records
8	Monitor quality and rebuild when configuration changes

12. Practical Best Practices

Collection design

- Use meaningful, stable names.
- Store purpose, owner, model, version, and source in metadata.
- Separate collections when security, lifecycle, model, or search behavior differs.

Data management

- Use deterministic or traceable IDs.
- Keep metadata schemas consistent.

- Validate record counts after batch ingestion.
- Use batch operations for large datasets.
- Test filters before update or deletion.

Retrieval quality

- Use one embedding model consistently for ingestion and querying.
- Select a distance metric compatible with the embedding model.
- Evaluate search quality using representative questions.
- Track source references so retrieved answers can be traced.

Production operations

- Persist the database instead of using an in-memory client for production.
- Back up source content and metadata.
- Plan migrations before changing embedding models or index configuration.
- Monitor document freshness, duplicate content, and failed ingestions.

Quick Revision Summary

Operation	Method	Key point
Create collection	create_collection()	Defines collection, metadata, embedding function, and index configuration.
Open collection	get_collection()	Connects to a collection that already exists.
Modify collection	modify()	Changes supported collection properties such as name and metadata.
Add records	add()	Stores documents, metadata, IDs, and generated embeddings.
Retrieve records	get()	Returns a dictionary; embeddings must be explicitly included.
Update records	update()	Modifies records and re-embeds changed document text.
Delete records	delete()	Deletes by ID, metadata filter, or both.

Knowledge-Check Questions with Answers

1. What is a collection in Chroma DB?

Answer: A collection is a container for related documents, embeddings, metadata, and unique IDs.

2. Which method creates a collection?

Answer: create_collection().

3. Which method connects to an existing collection?

Answer: get_collection().

4. Why is metadata useful?

Answer: It describes and classifies collections and records, supporting organization, filtering, lifecycle management, and traceability.

5. What does every document need when it is added?

Answer: A unique ID within the collection.

6. Are embeddings displayed by get() by default?

Answer: No. Request them explicitly with include=["embeddings"].

7. What happens when document text is updated?

Answer: Chroma DB re-computes the embedding and refreshes the indexed representation.

8. How can records be deleted?

Answer: By IDs, metadata filters, or a combination of both.

9. What is HNSW?

Answer: Hierarchical Navigable Small World, an approximate nearest-neighbor algorithm used for fast vector search.

10. Which distance options are discussed?

Answer: L2, cosine, and inner product.

11. Which metric is commonly selected for semantic text search?

Answer: Cosine distance is commonly used, provided it is appropriate for the selected embedding model.

12. Can the distance metric be changed directly on a populated collection?

Answer: Not as a simple modification. The collection generally needs to be recreated or cloned and rebuilt.

Brainstorming and Scenario Questions with Model Answers

These questions go beyond memorizing methods. They are designed to support design discussions, interviews, labs, and architecture reviews.

1. You must build a RAG assistant for policies, product manuals, and support tickets. Would you use one collection or several?

Answer: Use separate collections when the datasets have different access controls, owners, refresh cycles, embedding models, retention rules, or retrieval behavior. A single collection can work when governance and search behavior are uniform and metadata reliably distinguishes sources. The final decision should be driven by security boundaries and operational lifecycle, not only convenience.

2. How would you design document IDs so repeated ingestion does not create duplicates?

Answer: Use deterministic IDs derived from a stable source identifier, such as repository ID plus document path and chunk number. Before adding, check whether the ID exists and then update or upsert according to the ingestion strategy. Keep the original source key in metadata for traceability.

3. A team changes from one embedding model to another. What migration plan would you recommend?

Answer: Create a new collection configured for the new model, reprocess source documents, generate new embeddings, run side-by-side retrieval tests, validate result quality, then switch consumers after acceptance. Keep rollback capability until the new collection is proven.

4. Why might retrieval quality become poor even though all documents were added successfully?

Answer: Possible causes include mismatched ingestion and query embedding models, weak chunking, stale content, duplicate passages, insufficient metadata filtering, an unsuitable distance metric, or questions that do not match the indexed content. Successful insertion confirms storage, not retrieval quality.

5. How would you safely delete all documents from a specific source version?

Answer: First retrieve documents using the intended metadata filter and review the returned IDs and count. Back up source references if required. Then delete using the same filter, retrieve again to confirm removal, and record the operation for audit purposes.

6. What metadata would you include for enterprise documents?

Answer: Typical fields include source_system, source_id, document_type, owner, department, version, created_date, modified_date, language, security_classification, access_scope, chunk_number, and checksum. Use only fields that can be maintained consistently.

7. When should embeddings be included in a get() response?

Answer: Include them for debugging, migration, validation, export, or vector-level analysis. Avoid requesting them in routine application responses because vectors are large and generally not useful to end users.

8. How would you verify that an update really changed retrieval behavior?

Answer: Retrieve the record by ID to confirm the new text and metadata, then run representative semantic queries and compare ranking before and after the update. Validate that the revised passage appears for the expected questions without degrading unrelated results.

9. What risks arise from allowing unrestricted metadata?

Answer: Inconsistent naming, mixed data types, missing keys, sensitive data leakage, and filters that behave unpredictably. Define a metadata schema, validation rules, allowed values, and

ownership even when the database technically accepts flexible key-value pairs.

10. How would you choose between L2, cosine, and inner product?

Answer: Start with the embedding model guidance. Use cosine when vector direction is the important signal and the model is designed for cosine similarity. Use inner product when the model and ranking objective are trained for dot product. Use L2 when Euclidean geometry is appropriate. Confirm the choice through retrieval evaluation, not assumption alone.

11. A collection contains millions of records. Why is cloning expensive?

Answer: Cloning may require reading the full dataset, generating or copying embeddings, rebuilding the HNSW index, writing a second large data structure, and validating the result. The cost grows with collection size and can require substantial CPU, memory, storage, and operational coordination.

12. What controls would you add before using Chroma DB in a banking knowledge assistant?

Answer: Separate data by access boundary, enforce authentication and authorization in the application layer, prevent secrets and customer data from being indexed without approval, preserve source citations, log ingestion and deletion, encrypt storage and transport, test prompt-injection resistance, define retention rules, and require human validation for high-impact responses.

13. How can metadata improve retrieval beyond simple organization?

Answer: Metadata can restrict search to an approved business unit, language, version, document type, date range, or security scope. This reduces irrelevant candidates and helps enforce contextual boundaries before semantic ranking.

14. If two documents use the same ID, what design problem does this reveal?

Answer: The ingestion pipeline lacks a reliable identity strategy. IDs should uniquely and predictably represent records. Define whether identity applies to a whole document, a version, or an individual chunk, and implement collision checks.

15. What should be measured in a vector-search test set?

Answer: Measure whether relevant passages appear in the top results, ranking quality, coverage across question types, latency, failure cases, filter correctness, and source traceability. Include realistic domain questions and known expected evidence.

Final Summary

Chroma DB organizes vectorized information through collections containing documents, embeddings, metadata, and IDs. The main lifecycle methods are create_collection(), get_collection(), modify(), add(), get(), update(), and delete(). Document updates can trigger automatic re-embedding, while retrieval can return documents and metadata without exposing embeddings unless explicitly requested.

For similarity search, Chroma DB uses HNSW-based approximate nearest-neighbor indexing. The collection distance space, such as L2, cosine, or inner product, should be chosen when the collection is created and must align with the embedding model and retrieval objective. In production, reliable IDs, controlled metadata, safe migration, validation before deletion, and systematic retrieval-quality testing are as important as the database methods themselves.

Key takeaway
Successful vector applications depend on disciplined collection design, consistent embeddings, governed metadata, safe CRUD operations, and evidence-based retrieval evaluation.